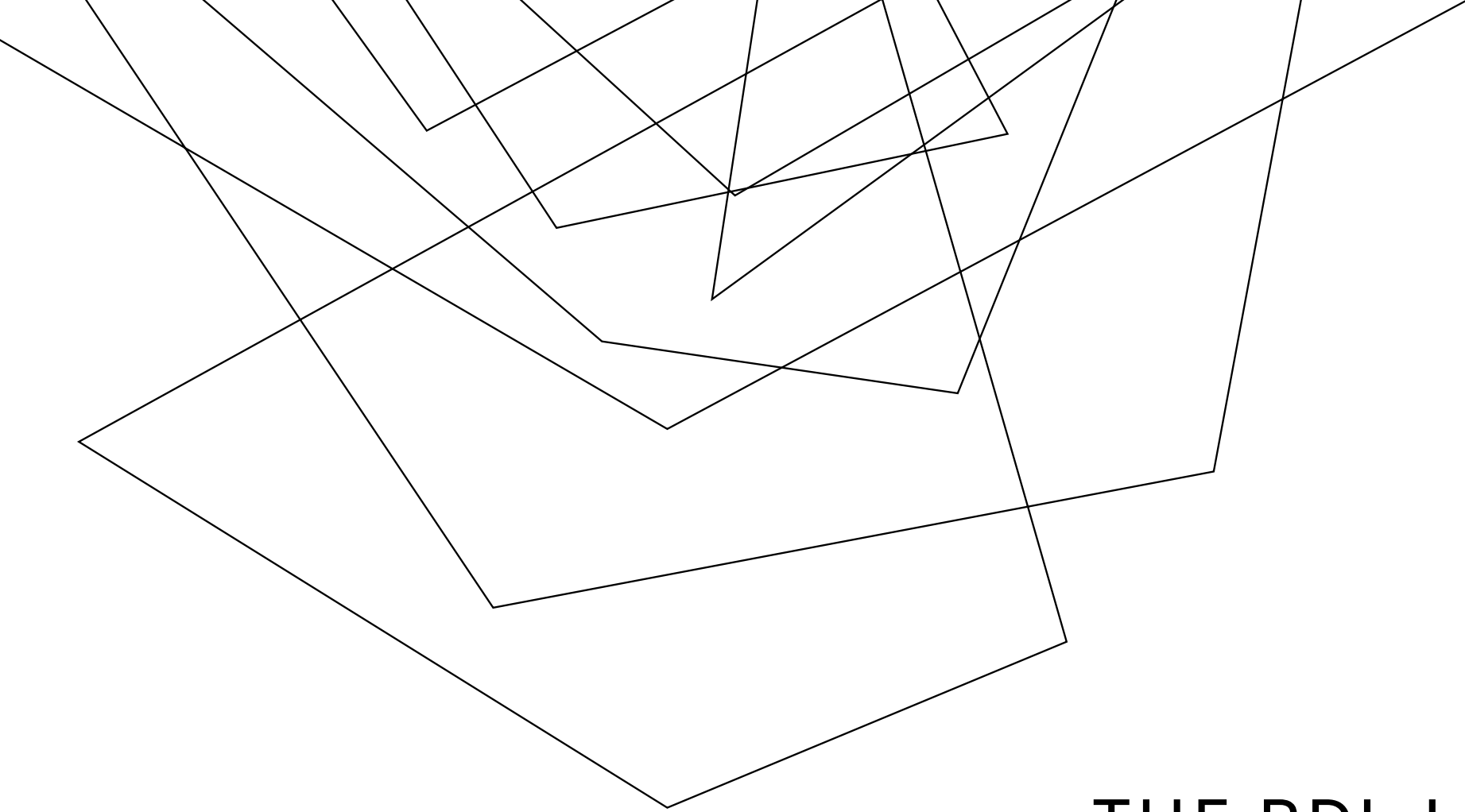




THE RDL LANGUAGE

AGENDA

Language Syntax

Scanner

Parser

Future



REGULAR DEFINITION LANGUAGE (RDL)

LANGUAGE SYNTAX

Overall

- Expressive and Models Thinking Patterns
- Define a new language with “lang”
- Define a new DFA with “dfa”
- “Symbol groups” to define a language
- Define transitions within a DFA using arrows

Example

```
test.rd
1  lang global = [a, b, c]
2  sublang(global) test = [a, b]
3
4  dfa MOD_3(global) {
5      start > 1
6      1 >[a] 2 >[a] 3 >[a] 1
7      1 <[b, c]
8      2 <[b, c]
9      3 <[b, c]
10     1 > end
11 }
```

TRANSITION DEFINITIONS

Example

- General Format:
 - Start State
 - Direction
 - On Symbol (All If Omitted)
 - End State
- Special **start** and **end** state
- “Linked” transitions

Special, Simple, and Linked

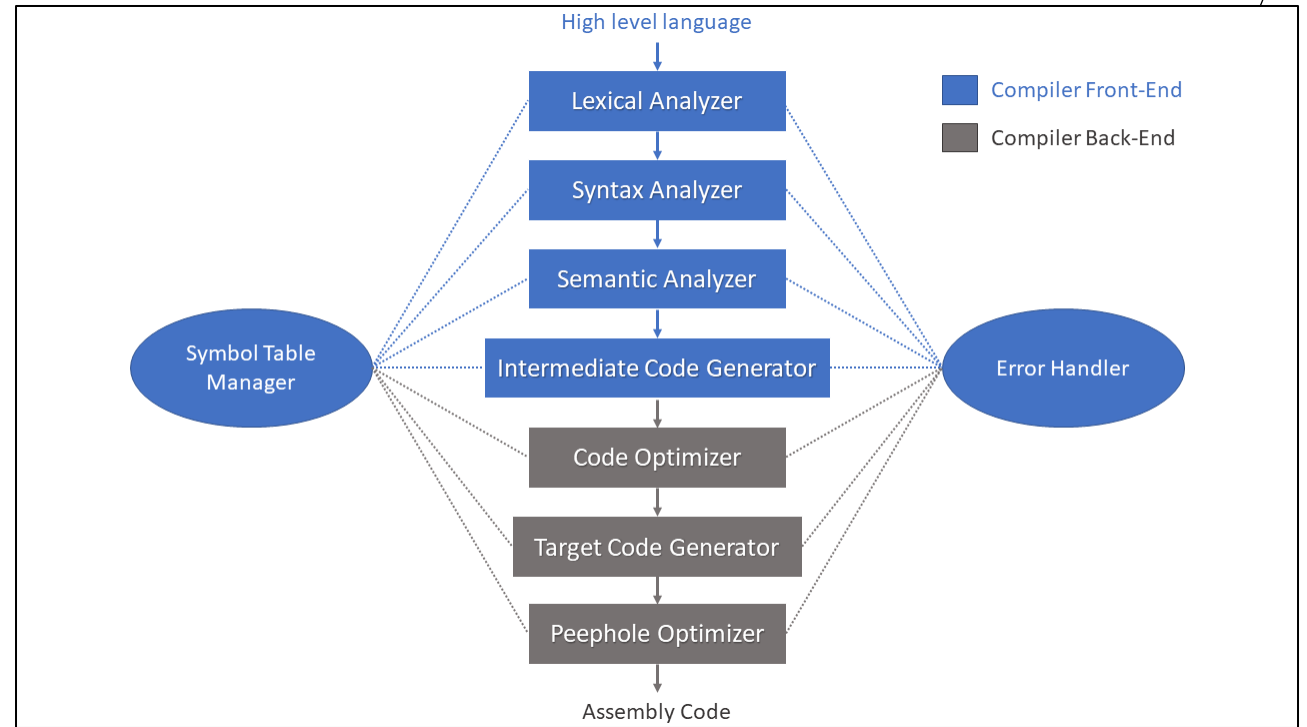
```
5      start > 1                # special
6      1 >[a] 2 >[a] 3 >[a] 1    # linked
7      1 <[b, c]                 # simple
8      2 <[b, c]                 # simple
9      3 <[b, c]                 # simple
10     1 > end                   # special
```

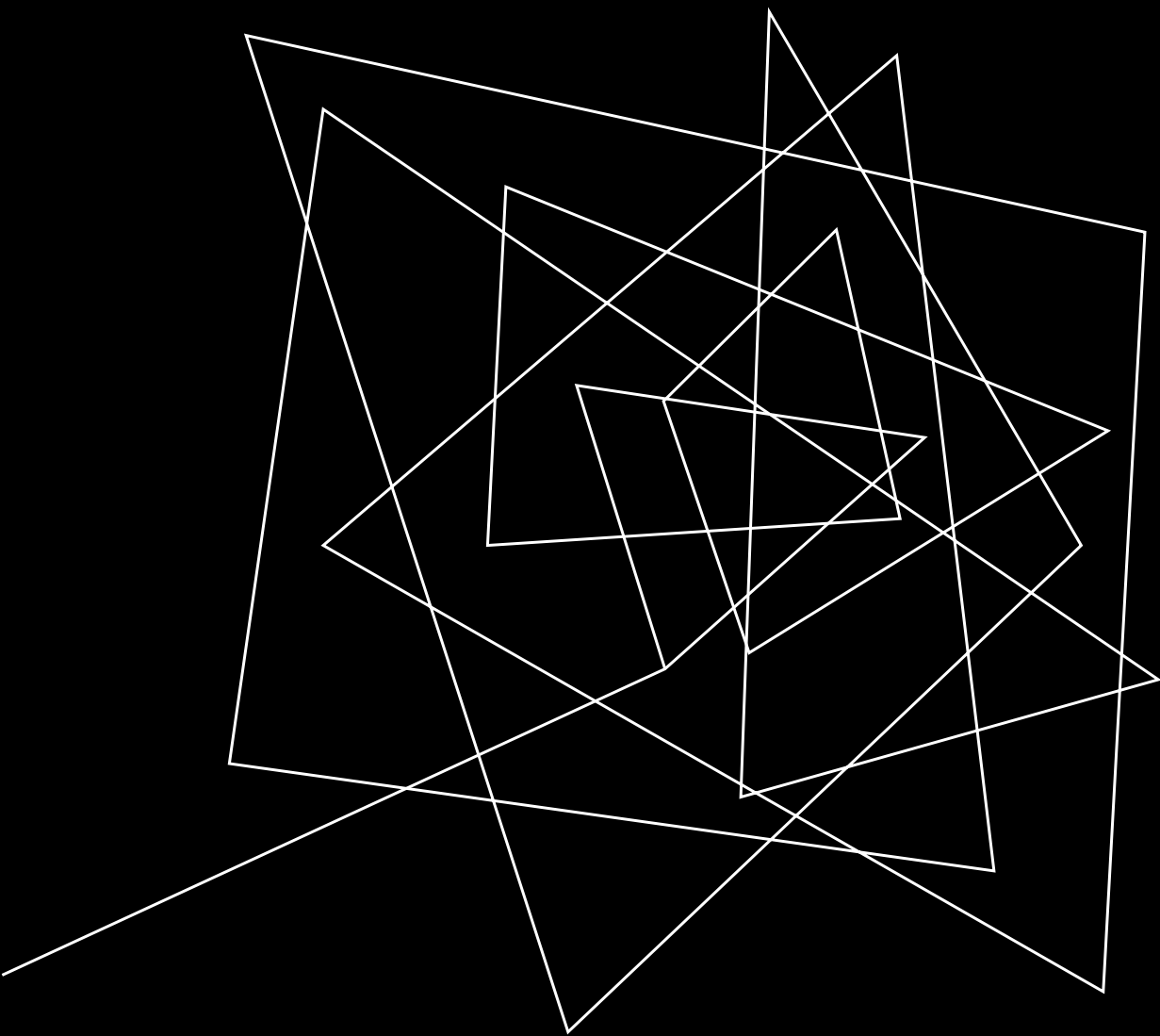
An abstract geometric design featuring two thin, dark grey lines that intersect on a light grey background. One line is oriented diagonally from the top-left towards the bottom-right, while the other is oriented from the top-right towards the bottom-left. The intersection point is located to the left of the text.

COMPILER IMPLEMENTATION

OVERALL DESIGN

- One-pass compiler
- Python as Intermediate Code
- Full-pass lexical analyzer and syntax analyzer

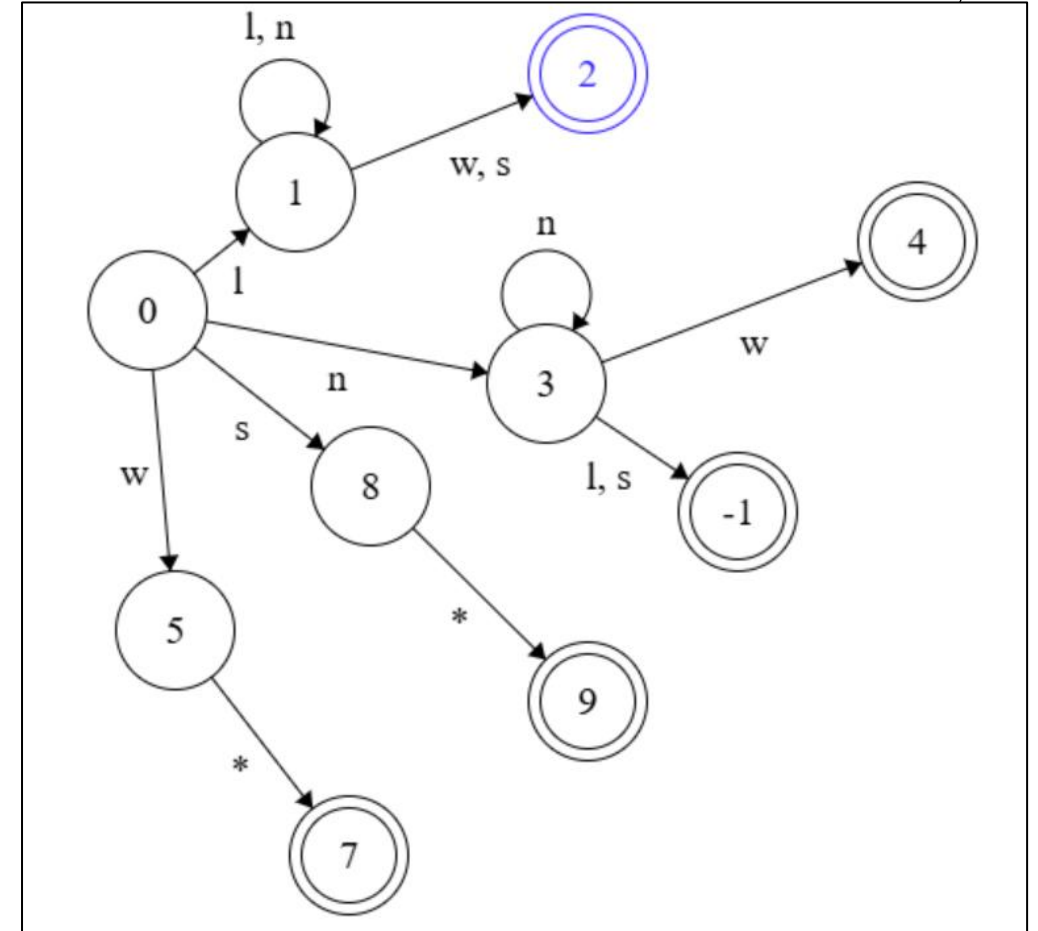




SCANNER

SCANNER (LEXICAL ANALYSIS)

- DFA to yield tokens
- 5 different token types:
KEYWORD, IDENTIFIER, NUMBER, SYMBOL, WHITESPACE, END
- Yield token at accepting state
- Error state at -1



SCANNER IMPLEMENTATION

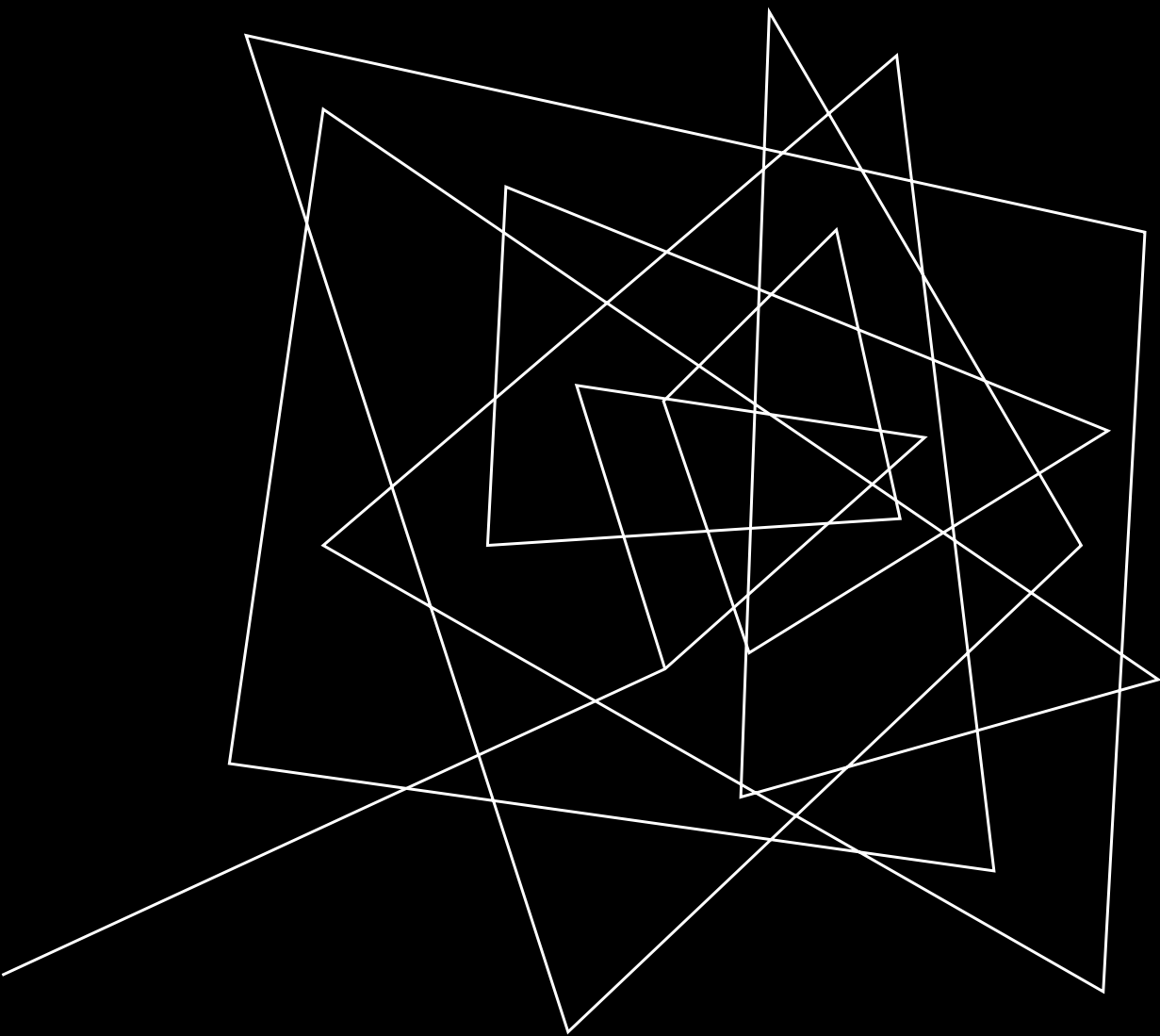
Description

- 2D array to traverse states
- Each language symbol is defined as a non-overlapping regex
- Goal of end compilation!!

Python Code

```
language = [whitespace, identifier, number, symbol]

transition_table: list[list[int | None]] = [
    # Input character
    # whitespace, letter, number, symbol
    [5, 1, 3, 6], # 0 start state
    [2, 1, 1, 2], # 1 first letter of identifier or keyword
    [None, None, None, None], # 2 yield identifier or keyword
    [4, -1, 3, -1], # 3 first letter of number
    [None, None, None, None], # 4 yield number
    [7, 7, 7, 7], # 5 first letter of whitespace
    [8, 8, 8, 8], # 6 first letter of symbol
    [None, None, None, None], # 7 yield whitespace
    [None, None, None, None], # 8 yield symbol
]
```

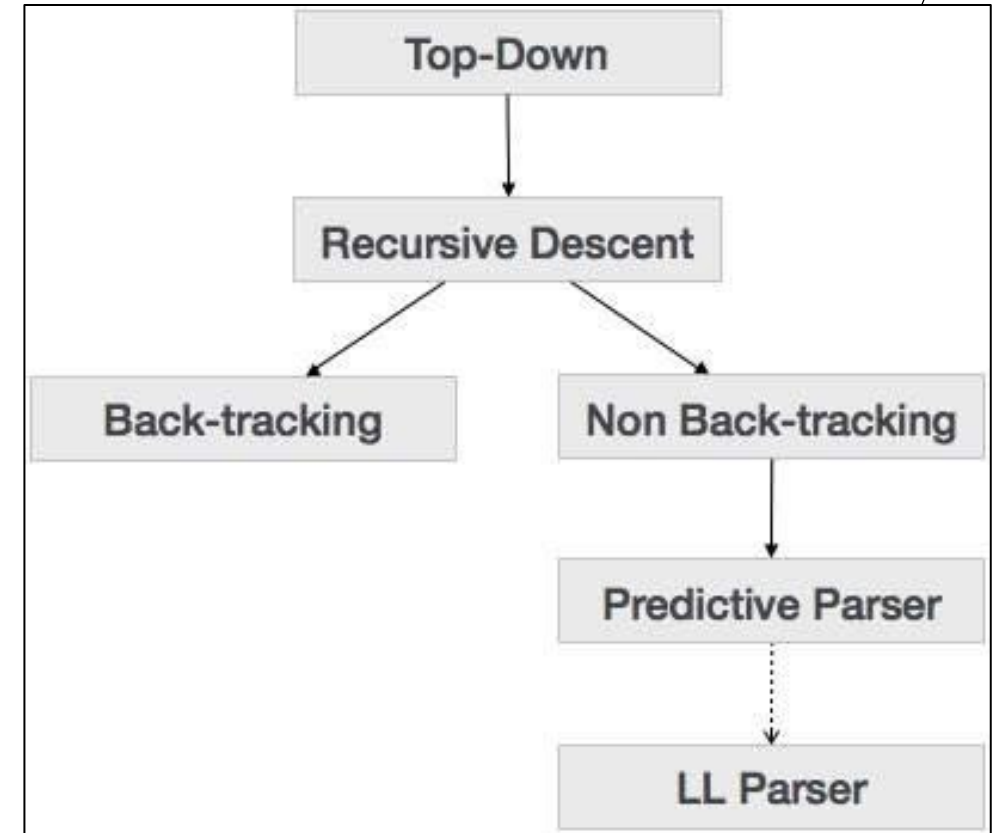


PARSER

It doesn't work

PARSER (SEMANTIC ANALYSIS)

- LL(1) Top-Down Parsing
- Left-to-Right, Left-Most-Derivation, 1-Look-Ahead
- Conditions:
 - No left recursion
 - Unambiguous grammar
 - Left factoring



Context Free Grammar

1-3 $S \rightarrow A ; \mid \text{for} (A ; C ; A) S \mid B$
 4-5 $A \rightarrow V = E \mid \epsilon$
 6-7 $C \rightarrow E \mid \epsilon$
 8 $E \rightarrow V$
 9 $V \rightarrow \text{id } X$
 10-11 $X \rightarrow \text{and } V \mid \epsilon$
 12 $B \rightarrow \{ L \}$
 13-14 $L \rightarrow S L \mid \epsilon$

Parse Table

	;	id	and	for	)	{	}	=	\$
S	A ;	A ;		for (A ; C ; A) S		B			synch
A	ϵ	V = E			ϵ				ϵ
C	ϵ	E							ϵ
E	synch	V			synch				synch
V	synch	id X			synch			synch	synch
X	ϵ		and V		ϵ			ϵ	ϵ
B	synch	synch		synch		{ L }			synch
L	S L	S L		S L		S L	ϵ		ϵ

A series of white, thin, overlapping geometric lines on a black background, creating a complex, abstract pattern on the left side of the slide.

THANK YOU

Daniel Liao

Alex Laria